1 Informe Técnico de Arquitectura
1.1 AXON Stack Flow — Sistema Web Full Stack
1.2 1. Resumen ejecutivo
1.3 2. Objetivos
1.4 3. Funcionalidades
1.5 4. Arquitectura general
1.6 5. Backend: arquitectura por capas
1.6.1 5.1 Controlador (`controllers/*.js`)
1.6.2 5.2 Servicio (`services/*.js`)
1.6.3 5.3 Repositorio (`repositories/*.js`)
1.6.4 5.4 Modelo (`models/*.js`)
1.6.5 5.5 DTO (`dto/userDTO.js`)
1.7 6. Base de datos
1.7.1 Entidades
1.7.2 Relaciones
1.7.3 Eliminación lógica
1.8 7. API REST (endpoints)
1.8.1 Autenticación (`/api/auth`)
1.8.2 Usuarios (`/api/users`) — admin para crear/editar/eliminar
1.8.3 Otros
1.9 8. Seguridad
1.10 9. Frontend (SPA)
1.10.1 Estructura
1.10.2 Características
1.11 10. Organización de carpetas (backend)
1.12 11. Instalación y uso
1.13 12. Despliegue (Docker)
1.14 13. Pruebas
1.15 14. Conclusiones
1.16 15. Referencias

1 Informe Técnico de Arquitectura

1.1 AXON Stack Flow — Sistema Web Full Stack

Campo	Valor
Proyecto	AXON Stack Flow
Versión	2.0
Fecha	2026-10-14
Stack	Node.js, Express, MySQL, HTML/CSS/JS
Arquitectura	Por capas (Controlador → Servicio → Repositorio → Modelo)

1.2 1. Resumen ejecutivo

AXON Stack Flow es una aplicación web completa que implementa la gestión de usuarios, autenticación segura con verificación por correo, monitoreo de endpoints y métricas del sistema. El backend está organizado en una arquitectura por capas que separa responsabilidades, facilitando el mantenimiento, las pruebas y el trabajo en equipo. El frontend es una SPA (Single Page Application) en HTML/CSS/JS puro con tema claro/oscuro, exportación CSV y gráficas.

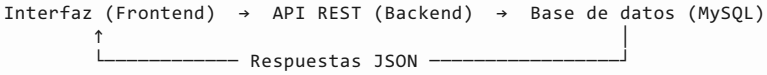
1.3 2. Objetivos

- Proveer un sistema de autenticación robusto basado en JWT + verificación por correo.
- Administrar usuarios con roles (`admin`/`user`) y eliminación lógica.
- Registrar y consultar métricas de uso (peticiones, latencia, códigos HTTP).
- Exponer una API REST documentada y organizada por recursos.
- Entregar una interfaz web profesional, responsive y accesible.
- Demostrar buenas prácticas: separación de capas, seguridad, pruebas, despliegue.

1.4 3. Funcionalidades

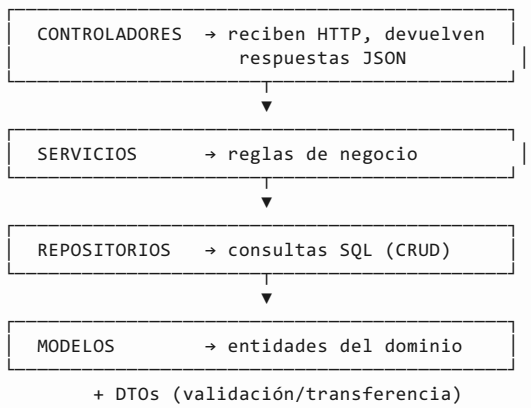
- Registro de usuarios con verificación de correo (código 6 dígitos, 5 min).
- Inicio de sesión en dos pasos (credenciales → código).
- Renovación de token (refresh token 7 días).
- Gestión de usuarios (CRUD, cambio de contraseña, eliminación lógica).
- Protección de usuarios demo (`is_sample`).
- Métricas del dashboard y actividad reciente.
- Gestión de endpoints monitorizados.
- Exportación CSV (usuarios y logs).
- Tema claro/oscuro persistente.

1.5 4. Arquitectura general



Componente	Tecnología	Responsabilidad
Frontend	HTML/CSS/JS (SPA)	Presentación y captura de acciones
Backend	Node.js + Express	Reglas de negocio y HTTP
API	REST (JSON)	Comunicación entre capas
BD	MySQL 8	Persistencia
Tiempo real	Socket.IO	Conexión socket básica

1.6 5. Backend: arquitectura por capas



1.6.1 5.1 Controlador (`controllers/*.js`)

Recibe la solicitud HTTP, valida presencia básica y delega al servicio.

```
async login(req, res, next) {
  const { email, password } = req.body || {};
  const result = await authService.login(email, password, req.ip);
  res.json(result);
}
```

1.6.2 5.2 Servicio (`services/*.js`)

Contiene reglas de negocio: validación de email, rate limiting, hashing, generación de códigos, emisión de tokens.

1.6.3 5.3 Repositorio (`repositories/*.js`)

Operaciones CRUD contra MySQL (`findByEmail`, `create`, `update`, `softDelete`).

1.6.4 5.4 Modelo (`models/*.js`)

Representación de entidades del dominio (`UserModel` con `isActive()`, `isAdmin()`).

1.6.5 5.5 DTO (`dto/userDTO.js`)

Valida tamaño/formato de entrada y define la salida (`UserResponseDTO` excluye `password_hash`).

1.7 6. Base de datos

1.7.1 Entidades

Tabla	Campos principales
`users`	id, name, email, password_hash, role, email_verified, created_date, updated_date, created_by_id, is_sample
`endpoints`	id, name, path, method, description, created_at
`request_logs`	id, method, path, status_code, latency_ms, source, user_id, created_at
`login_verification_codes`	id, user_id, email, purpose, code_hash, expires_at, attempts, used, created_at

1.7.2 Relaciones

- `users.created_by_id → users.id` (NULL permitido).
- `request_logs.user_id → users.id` (SET NULL).
- `login_verification_codes.user_id → users.id` (CASCADE).

1.7.3 Eliminación lógica

Los usuarios se marcan con `role = 'inactive'` y no se borran físicamente. Los demo (`is_sample = 1`) están protegidos.

1.8 7. API REST (endpoints)

1.8.1 Autenticación (`/api/auth`)

Método	Endpoint	Entrada	Respuesta
POST	`/login`	`{ email, password }`	`{ requiresVerification, email }`
POST	`/verify-code`	`{ email, code }`	`{ token, refreshToken, user }`
POST	`/register`	`{ name, email, password }`	`{ requiresVerification, email }`
POST	`/verify-registration`	`{ email, code }`	`{ verified: true }`
POST	`/resend-code`	`{ email }`	`{ message }`
POST	`/resend-registration`	`{ email }`	`{ message }`
POST	`/refresh-token`	`{ refreshToken }`	`{ token, refreshToken, user }`

1.8.2 Usuarios (`/api/users`) — admin para crear/editar/eliminar

Método	Endpoint	Descripción
GET	`/`	Listar usuarios activos (paginado)
GET	`/:id`	Obtener usuario
POST	`/`	Crear usuario
PUT	`/:id`	Actualizar (name, email, role)
PATCH	`/:id/password`	Cambiar contraseña
DELETE	`/:id`	Eliminación lógica

1.8.3 Otros

- `GET /api/metrics/dashboard` → resumen + actividad reciente.
- `GET/POST/DELETE /api/axon/endpoints` → endpoints monitorizados.
- `GET /api/health` → estado del servicio.

Códigos HTTP utilizados: 200, 201, 400, 401, 403, 404, 409, 429, 500.

1.9 8. Seguridad

Medida	Detalle
Contraseñas	bcryptjs, salt rounds 10
JWT acceso	8 horas
Refresh token	7 días, firmado con secreto independiente
Rate limiting IP	5 intentos / 15 min
Códigos 2FA	6 dígitos, expiran 5 min, 5 intentos
Validación	Regex en frontend y backend
Eliminación lógica	`role = 'inactive'`
CORS	Origen del frontend configurado
Middleware	`helmet`, validación, errorHandler central

1.10 9. Frontend (SPA)

1.10.1 Estructura

```
frontend/
├── css/styles.css        # variables, componentes, temas
├── js/
│   ├── core/             # config, api, auth, ui
│   ├── pages/            # vistas por pantalla
│   └── app.js            # router y shell (sidebar)
├── index.html
└── README.md
```

1.10.2 Características

- Router basado en hash (`#/dashboard`, `#/users`, …).
- Cliente HTTP con renovación automática de token.
- Tema claro/oscuro persistente.
- Gráficas con Chart.js.
- Exportación CSV.
- Responsive y accesible (focus, aria).

1.11 10. Organización de carpetas (backend)

```
backend/src/
├── config/        database.js, validateEnv.js
├── controllers/   authController, userController, metricsController,
axonController
├── services/      authService, userService, emailService, metricsService,
axonService
├── repositories/  userRepository, loginCodeRepository,
endpointRepository, logRepository
├── models/        userModel, endpointModel, logModel, loginCodeModel
├── dto/           userDTO (CreateUserDTO, UpdateUserDTO, UserResponseDTO,
LoginDTO)
├── middleware/    auth, errorHandler, rateLimiter, requestLogger
├── routes/        authRoutes, userRoutes, metricsRoutes, axonRoutes,
healthRoutes
├── socket/        socket.js
├── app.js, server.js, dbInit.js, seed.js
├── tests/         dto.test.js, api.test.js
└── README.md
```

1.12 11. Instalación y uso

Ver `docs/GUIA_USO.md` y `README.md`. Comandos rápidos:

```
cd backend
npm install
npm run db:init
npm run seed
npm start           # http://localhost:4000
npm test            # pruebas

node scripts/serve-frontend.js   # http://localhost:5500
```

1.13 12. Despliegue (Docker)

```
docker compose up -d --build
```

Servicios: MySQL 8, backend (Node) y frontend (nginx). Frontend en `http://localhost:8080`.

1.14 13. Pruebas

`npm test` ejecuta 7 suites (node:test): 1. `GET /api/health` → 200. 2. Login inválido → 401. 3. Ruta protegida sin token → 401. 4. Login válido → solicita verificación. 5–7. Validación de DTOs.

1.15 14. Conclusiones

El sistema implementa una arquitectura por capas clara y escalable, separando controladores, servicios, repositorios y modelos. La seguridad sigue buenas prácticas (hashing, JWT, rate limiting, códigos). El frontend es una SPA desacoplada, profesional y responsive. El proyecto incluye pruebas automatizadas y despliegue con Docker.

1.16 15. Referencias

- Documentación de Express.js — https://expressjs.com
- MySQL 8 Reference Manual — https://dev.mysql.com/doc/
- JSON Web Tokens (RFC 7519) — https://datatracker.ietf.org/doc/html/rfc7519
- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- Chart.js — https://www.chartjs.org